MINI INVENTORY

Component 1: Build and Continuous Integration

Java REST API, Git workflow, Maven testing and automatic Jenkins SCM builds

Student	Emaad Mirza
USN	1ms24cy020
Execution date	3 November 2026
Platform	Windows and Docker Desktop
Repository	https://github.com/EmaadMirza/Inventory-devops

Evidence checkpoint: Backend execution is verified. Jenkins browser screenshots are pending because browser automation could not verify the Windows browser URL. This report is a review draft until those screenshots are inserted; terminal/API records below are genuine and are not described as browser screenshots.

This report records observed execution results. Configuration listings are the reviewed assignment source. Screenshots originate from actual terminal or Jenkins sessions. No coordinator invitation is claimed.

Abstract

A small inventory REST service was developed in Java and packaged as an executable Maven JAR. Git preserved a real feature branch and merge. Jenkins checked out the GitHub repository, ran automated tests, packaged the service and archived its artifact. A later documentation push was detected automatically by SCM polling. All seven automated tests passed; live JAR and Docker HTTP checks returned the expected success and error responses.

Introduction

Manual builds are difficult to reproduce and can distribute untested changes. This assignment connects version control, automated validation and artifact publication around one inspectable service. The inventory application supplies product creation, lookup, listing and stock updates. Its low-stock filter is an enhancement with a clear replenishment use case.

Implementation: environment and review

The work used an 8 GB Windows laptop with Docker Desktop 4.91.0 and Docker Engine 29.8.0. Initial inspection found roughly 1.8 GB free host RAM, 18.7 GB free on C:, four cached unrelated images and no containers. None of those images supplied the required Java/Jenkins runtimes. Downloads and running services were staged sequentially. Existing projects and the earlier Jenkins assignment were retained. Resource snapshots are in evidence/resources.txt.

All 24 prepared files were reviewed before execution. The root Docker ignore file incorrectly excluded the Ansible target entrypoint and was corrected. Jenkins was moved from its prepared Java 17 tag to lts-jdk21 to meet current requirements. The image actually ran Jenkins 2.580.2 on Java 21.0.13. Port 8081 was occupied, so this isolated lab uses localhost:18081. Jenkins has one executor and a 1 GiB container limit; its Java heap is capped at 512 MiB.

REST API design

Request	Observed behavior
GET /health	200; UP and instance identity
GET /products	200; product list
POST /products	201; validated name and nonnegative integer stock
GET /products/{id}	200 for existing product, 404 otherwise
PUT /products/{id}/stock	200 update; 400 negative stock; 404 missing product
GET /products?maxStock=N	200; products whose stock is at most N
Invalid input/method	400 for malformed/product input; 405 for unsupported method

Inventory methods synchronize access to immutable Product records. JSON bodies are limited to 8 KiB, and the HTTP server uses a bounded worker queue. Maven compiles with release 17 and shades Gson into target/mini-inventory.jar. The service can therefore run without a separate dependency classpath.

Git feature branch and real merge

The baseline commit was a783f65. Branch feature/low-stock-filter added GET /products?maxStock=N and a real HTTP regression test in commit 8ae847a. Main merged it with --no-ff in commit e7dc7ea, preserving both parents. Main and the feature branch were pushed to the new Inventory-devops repository. No history rewrite or branch deletion was used.

Jenkins automatic SCM pipeline

The Pipeline from SCM job uses the public GitHub repository, branch */main and Jenkinsfile. Build #1 was intentionally started once to register polling. A real documentation commit a61cb7c was then pushed. The configured once-per-minute poll found that revision and scheduled build #2. Its recorded cause is hudson.triggers.SCMTrigger$SCMTriggerCause, with description 'Started by an SCM change'. No build request was sent for build #2.

The pipeline checks out source, runs mvn clean test, and packages only after the test stage succeeds. Its post block publishes Surefire XML through JUnit and archives the executable JAR with fingerprinting. A timeout, retention policy and disabled concurrent builds keep the lab bounded. Local writes require the generated emaad account; read-only results are accessible on the loopback-only Jenkins port. Credentials are excluded from Git and the package.

Enhancement rationale

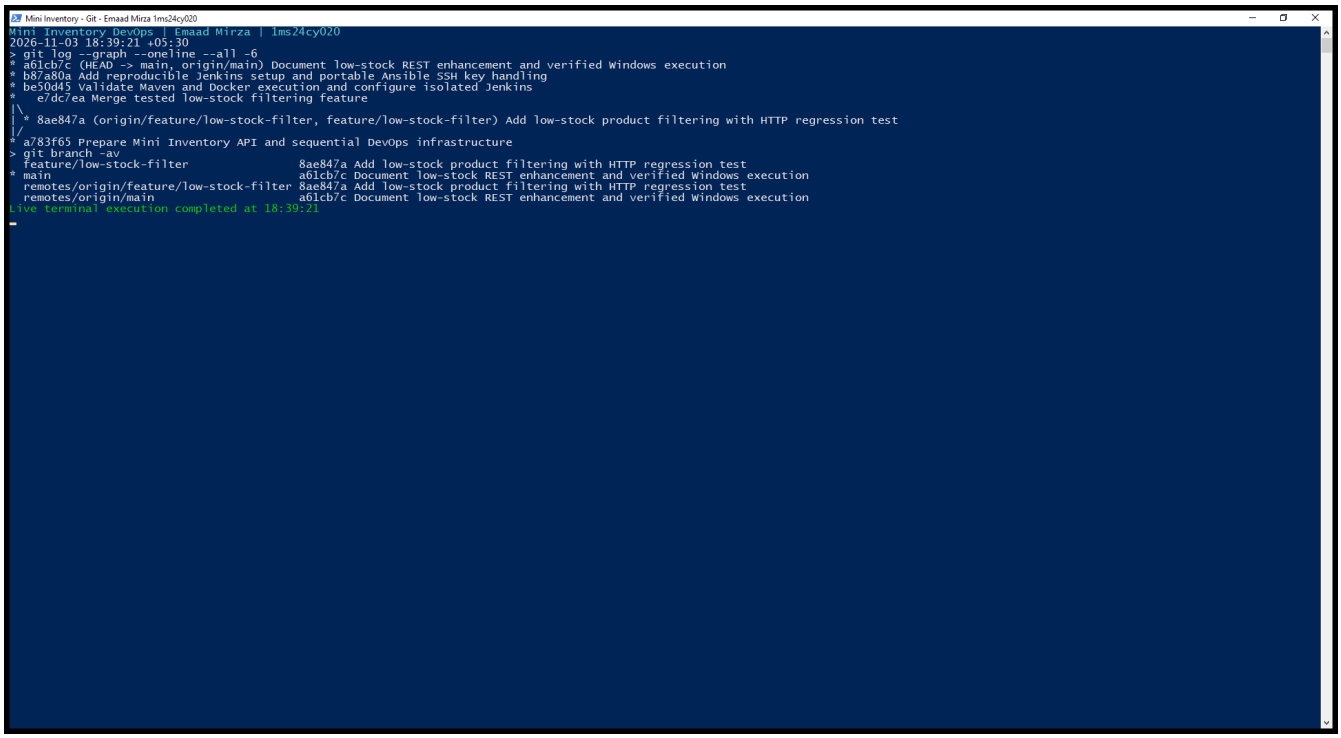
The low-stock filter makes the demo useful for identifying items requiring replenishment. It operates on a snapshot of inventory without mutating products. Its regression test creates products above and below a threshold, checks the filtered response, verifies the full inventory remains intact, and rejects a negative threshold. This is the actual change committed and merged on the feature branch. Automated HTTP tests extend the original three inventory tests to exercise the application boundary.

Actual results

Check	Actual result
Maven clean verify	BUILD SUCCESS; 7 tests; 0 failures/errors/skips
Executable JAR HTTP	9 checks passed, including 201/400/404/405 behavior
Git	Real merge e7dc7ea; main and feature branch retained locally/remotely
Jenkins #1	SUCCESS; JUnit results and JAR archived
Jenkins #2	SUCCESS; Started by an SCM change; 7 tests, 0 failures
Archived JAR	323,714 bytes; target/mini-inventory.jar
Artifact SHA256	d1408930d037fcf1128500f77e0ef21082a740746284db78427ff7a12c8f2170

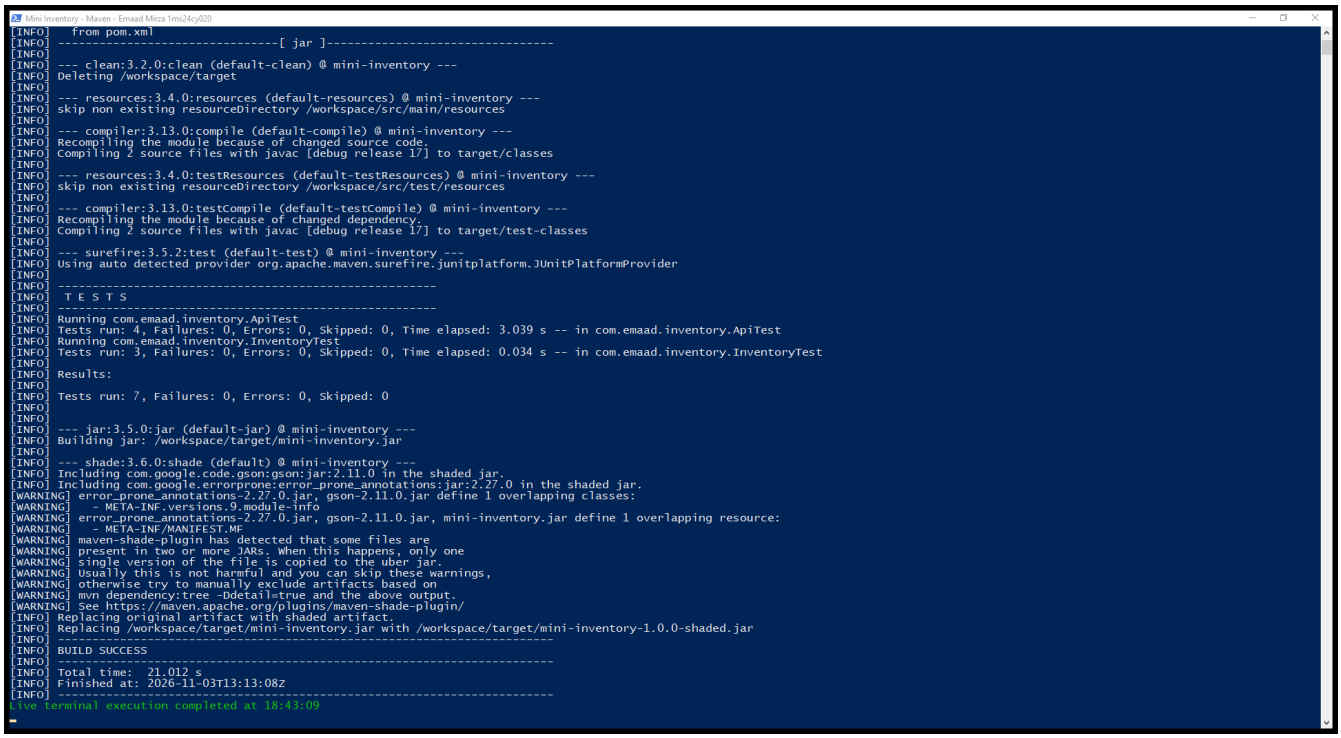
The local Maven/JDK 17 JAR and the Jenkins/JDK 21 JAR need not have identical bytes because build timestamps and compiler versions differ. The archived JAR was downloaded through Jenkins and its actual checksum recorded. The application image was separately built and checked; those results are also included in Component 2.

Evidence: real Git merge and preserved branches



Original screenshot: evidence/terminal-Git.png. Embedded without alteration; zoom to inspect command output.

Evidence: live Maven verification



Original screenshot: evidence/terminal-Maven.png. Embedded without alteration; zoom to inspect command output.

Conclusion

The API, executable artifact, feature workflow and automatic SCM-triggered CI were executed successfully. The results demonstrate tested changes flowing from GitHub to a published Jenkins artifact. Inventory remains process-local and is unsuitable for shared state across replicas; a production version should use a shared database and authenticated API access. The evidence checkpoint on the cover states any remaining browser screenshot limitation.

References

Apache Maven lifecycle. https://maven.apache.org/guides/introduction/introduction-to-the-lifecycle.html. Accessed 3 November 2026.

Jenkins Pipeline syntax and pollSCM. https://www.jenkins.io/doc/book/pipeline/syntax/. Accessed 3 November 2026.

Jenkins Java support. https://www.jenkins.io/doc/book/platform-information/support-policy-java/. Accessed 3 November 2026.

Git merge. https://git-scm.com/docs/git-merge. Accessed 3 November 2026.

Assignment repository and execution evidence. https://github.com/EmaadMirza/Inventory-devops. Accessed 3 November 2026.

Code and configuration listings

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion><groupId>com.emaad</groupId><artifactId>mini-
    inventory</artifactId><version>1.0.0</version>
  <properties><maven.compiler.release>17</maven.compiler.release><project.build.sourceEncoding>UTF-
    8</project.build.sourceEncoding></properties>
  <dependencies>
    <dependency><groupId>com.google.code.gson</groupId><artifactId>gson</artifactId><version>2.11.0</version
    ></dependency>
    <dependency><groupId>org.junit.jupiter</groupId><artifactId>junit-
    jupiter</artifactId><version>5.11.4</version><scope>test</scope></dependency>
  </dependencies>
  <build><finalName>mini-inventory</finalName><plugins>
    <plugin><groupId>org.apache.maven.plugins</groupId><artifactId>maven-compiler-
    plugin</artifactId><version>3.13.0</version></plugin>
    <plugin><groupId>org.apache.maven.plugins</groupId><artifactId>maven-surefire-
    plugin</artifactId><version>3.5.2</version></plugin>
    <plugin><groupId>org.apache.maven.plugins</groupId><artifactId>maven-shade-plugin</artifactId><version>3
    .6.0</version><executions><execution><phase>package</phase><goals><goal>shade</goal></goals><configurati
    on><createDependencyReducedPom>false</createDependencyReducedPom><transformers><transformer implementati
    on="org.apache.maven.plugins.shade.resource.ManifestResourceTransformer"><mainClass>com.emaad.inventory.
    App</mainClass></transformer></transformers></configuration></execution></executions></plugin>
  </plugins></build>
</project>
```

src/main/java/com/emaad/inventory/Inventory.java

```
package com.emaad.inventory;
import java.util.*;

/** One process owns one inventory. State is intentionally not shared across replicas. */
public final class Inventory {
    public record Product(long id, String name, int stock) {}
    private final Map<Long, Product> products = new LinkedHashMap<>();
    private long nextId = 1;
    public synchronized Product add(String name, int stock) {
        if (name == null || name.isBlank() || stock < 0) throw new IllegalArgumentException("Name is
    required and stock must be nonnegative");
        Product p = new Product(nextId++, name.trim(), stock); products.put(p.id(), p); return p;
    }
    public synchronized List<Product> list() { return List.copyOf(products.values()); }
    public synchronized Product find(long id) {
        Product product = products.get(id);
        if (product == null) throw new NoSuchElementException("Product not found");
        return product;
    }
    public synchronized Product setStock(long id, int stock) {
        if (stock < 0) throw new IllegalArgumentException("Stock must be nonnegative");
        Product old = products.get(id);
        if (old == null) throw new NoSuchElementException("Product not found");
        Product updated = new Product(id, old.name(), stock); products.put(id, updated); return updated;
    }
}
```

src/main/java/com/emaad/inventory/App.java

```
package com.emaad.inventory;
import com.google.gson.*;
import com.sun.net.httpserver.*;
import java.io.*;
import java.net.*;
import java.nio.charset.StandardCharsets;
import java.util.*;
import java.util.concurrent.*;

public final class App {
    private static final Gson JSON = new Gson();
    private static final int MAX_BODY = 8192;
    public static HttpServer start(int port, Inventory inventory) throws IOException {
        HttpServer server = HttpServer.create(new InetSocketAddress("0.0.0.0", port), 32);
```

```
        server.setExecutor(new ThreadPoolExecutor(2, 4, 30, TimeUnit.SECONDS,
            new ArrayBlockingQueue<>(64), new ThreadPoolExecutor.CallerRunsPolicy()));
        server.createContext("/", exchange -> {
            try {
                String path = exchange.getRequestURI().getPath();
                String method = exchange.getRequestMethod();
                if (path.equals("/health")) {
                    if (!method.equals("GET")) { send(exchange,405,Map.of("error","Use GET")); return; }
    send(exchange,200,Map.of("status","UP","instance",System.getenv().getOrDefault("HOSTNAME","local")));
                } else if (path.equals("/products")) {
                    if (method.equals("GET")) {
                        String query = exchange.getRequestURI().getQuery();
                        if (query == null) send(exchange,200,inventory.list());
                        else if (query.matches("maxStock=[0-9]+")) {
                            int maximum = Integer.parseInt(query.substring(9));
                            send(exchange,200,inventory.list().stream().filter(p -> p.stock() <=
    maximum).toList());
                        } else throw new IllegalArgumentException("Use maxStock=nonnegative integer");
                    }
                    else if (method.equals("POST")) {
                        JsonObject body = readBody(exchange);
                        if (!body.has("name") || !body.get("name").isJsonPrimitive() ||
    !body.getAsJsonPrimitive("name").isString()) throw new IllegalArgumentException("name must be a
    string");
                        send(exchange,201,inventory.add(body.get("name").getAsString(),stock(body)));
                    } else send(exchange,405,Map.of("error","Use GET or POST"));
                } else if (path.matches("/products/[0-9]+")) {
                    if (!method.equals("GET")) { send(exchange,405,Map.of("error","Use GET")); return; }
                    send(exchange,200,inventory.find(Long.parseLong(path.split("/")[2])));
                } else if (path.matches("/products/[0-9]+/stock")) {
                    if (!method.equals("PUT")) { send(exchange,405,Map.of("error","Use PUT")); return; }
                    long id = Long.parseLong(path.split("/")[2]);
                    send(exchange,200,inventory.setStock(id,stock(readBody(exchange))));
                } else send(exchange,404,Map.of("error","Not found"));
            } catch (NoSuchElementException e) { send(exchange,404,Map.of("error",e.getMessage())); }
              catch (IllegalArgumentException | JsonParseException | ArithmeticException e) {
    send(exchange,400,Map.of("error","Invalid JSON or product data")); }
              catch (Exception e) { send(exchange,500,Map.of("error","Internal server error")); }
            finally { exchange.close(); }
        });
        server.start(); return server;
    }
    private static JsonObject readBody(HttpExchange exchange) throws IOException {
        byte[] bytes = exchange.getRequestBody().readNBytes(MAX_BODY + 1);
        if (bytes.length > MAX_BODY) throw new IllegalArgumentException("Request too large");
        JsonElement parsed = JsonParser.parseString(new String(bytes, StandardCharsets.UTF_8));
        if (!parsed.isJsonObject()) throw new IllegalArgumentException("Expected JSON object");
        return parsed.getAsJsonObject();
    }
    private static int stock(JsonObject body) {
        JsonElement value = body.get("stock");
        if (value == null || !value.isJsonPrimitive() || !value.getAsJsonPrimitive().isNumber()) throw new
    IllegalArgumentException("stock must be an integer");
        return value.getAsBigDecimal().intValueExact();
    }
    private static void send(HttpExchange exchange, int status, Object body) throws IOException {
        byte[] bytes = JSON.toJson(body).getBytes(StandardCharsets.UTF_8);
        exchange.getResponseHeaders().set("Content-Type","application/json; charset=utf-8");
        exchange.sendResponseHeaders(status,bytes.length); exchange.getResponseBody().write(bytes);
    }
    public static void main(String[] args) throws Exception {
        Inventory inventory = new Inventory(); inventory.add("Notebook",10); inventory.add("Pen",25);
        int port = Integer.parseInt(System.getenv().getOrDefault("APP_PORT","8080"));
        HttpServer server = start(port,inventory);
        Runtime.getRuntime().addShutdownHook(new Thread(() -> { server.stop(1);
    ((ExecutorService)server.getExecutor()).shutdownNow(); }));
        System.out.println("Mini Inventory API listening on " + port);
    }
}
```

src/test/java/com/emaad/inventory/InventoryTest.java

```
package com.emaad.inventory;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
class InventoryTest {
    @Test void createsAndListsProducts() {
```

```
        Inventory i = new Inventory(); var p=i.add("Notebook",10);
        assertEquals("Notebook",p.name()); assertEquals(10,i.list().get(0).stock());
    }
    @Test void updatesStockAndRejectsMissingProduct() {
        Inventory i=new Inventory(); var p=i.add("Pen",5);
        assertEquals(8,i.setStock(p.id(),8).stock());
        assertThrows(java.util.NoSuchElementException.class,()->i.setStock(999,1));
    }
    @Test void rejectsInvalidProductsAndStock() {
        Inventory i=new Inventory();
        assertThrows(IllegalArgumentException.class,()->i.add(" ",1));
        assertThrows(IllegalArgumentException.class,()->i.add("Pen",-1));
        var p=i.add("Pen",0);
        assertThrows(IllegalArgumentException.class,()->i.setStock(p.id(),-2));
        assertEquals(0,i.list().get(0).stock());
    }
}
```

src/test/java/com/emaad/inventory/ApiTest.java

```
package com.emaad.inventory;

import com.sun.net.httpserver.HttpServer;
import java.net.URI;
import java.net.http.*;
import java.util.concurrent.ExecutorService;
import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

class ApiTest {
    private HttpServer server;
    private String base;
    private final HttpClient client = HttpClient.newHttpClient();
    @BeforeEach void start() throws Exception {
        server = App.start(0, new Inventory());
        base = "http://127.0.0.1:" + server.getAddress().getPort();
    }
    @AfterEach void stop() {
        server.stop(0);
        ((ExecutorService) server.getExecutor()).shutdownNow();
    }
    private HttpResponse<String> request(String method, String path, String json) throws Exception {
        return client.send(HttpRequest.newBuilder(URI.create(base + path))
            .header("Content-Type", "application/json")
            .method(method, HttpRequest.BodyPublishers.ofString(json)).build(),
            HttpResponse.BodyHandlers.ofString());
    }
    @Test void createsFindsAndUpdatesProductOverHttp() throws Exception {
        assertEquals(201, request("POST", "/products", "{\"name\":\"Pen\",\"stock\":2}").statusCode());
        var found = request("GET", "/products/1", "");
        assertEquals(200, found.statusCode());
        assertTrue(found.body().contains("Pen"));
        assertEquals(200, request("PUT", "/products/1/stock", "{\"stock\":8}").statusCode());
        assertTrue(request("GET", "/products/1", "").body().contains("\"stock\":8"));
        assertEquals(404, request("GET", "/products/999", "").statusCode());
        assertEquals(405, request("POST", "/products/1", "{}").statusCode());
    }
    @Test void rejectsMalformedFractionalNegativeAndOversizedInput() throws Exception {
        for (String body : new String[]{"{", "null", "[]", "{\"name\":null,\"stock\":1}",
            "{\"name\":\"Pen\",\"stock\":1.5}", "{\"name\":\"Pen\",\"stock\":-1}", " ".repeat(8193)}) {
            assertEquals(400, request("POST", "/products", body).statusCode(), body.substring(0,
    Math.min(80, body.length())));
        }
    }
    @Test void filtersLowStockWithoutChangingInventory() throws Exception {
        request("POST", "/products", "{\"name\":\"Pen\",\"stock\":2}");
        request("POST", "/products", "{\"name\":\"Notebook\",\"stock\":10}");
        var filtered = request("GET", "/products?maxStock=2", "");
        assertEquals(200, filtered.statusCode());
        assertTrue(filtered.body().contains("Pen"));
        assertFalse(filtered.body().contains("Notebook"));
        assertTrue(request("GET", "/products", "").body().contains("Notebook"));
        assertEquals(400, request("GET", "/products?maxStock=-1", "").statusCode());
    }
    @Test void reportsHealthAndRejectsUnknownRoute() throws Exception {
        assertEquals(200, request("GET", "/health", "").statusCode());
        assertEquals(404, request("GET", "/unknown", "").statusCode());
    }
```

```
}
```

Jenkinsfile

```
pipeline {
  agent any
  triggers { pollSCM('* * * * *') }
  options {
    timestamps()
    timeout(time: 10, unit: 'MINUTES')
    buildDiscarder(logRotator(numToKeepStr: '10'))
    disableConcurrentBuilds()
  }
  stages {
    stage('Checkout') { steps { checkout scm } }
    stage('Test') { steps { sh 'mvn -B -ntp clean test' } }
    stage('Package') { steps { sh 'mvn -B -ntp -DskipTests package' } }
  }
  post {
    always { junit testResults: 'target/surefire-reports/*.xml', allowEmptyResults: false }
    success { archiveArtifacts artifacts: 'target/mini-inventory.jar', fingerprint: true }
  }
}
```

infra/jenkins/Dockerfile

```
FROM jenkins/jenkins:lts-jdk21
USER root
RUN apt-get update && apt-get install -y --no-install-recommends maven && rm -rf /var/lib/apt/lists/*
USER jenkins
RUN jenkins-plugin-cli --plugins workflow-aggregator git junit timestamper
```

infra/jenkins/job.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<flow-definition plugin="workflow-job">
  <description>Emaad Mirza - 1ms24cy020 - Mini Inventory SCM pipeline</description>
  <keepDependencies>false</keepDependencies>
  <properties/>
  <definition class="org.jenkinsci.plugins.workflow.cps.CpsScmFlowDefinition" plugin="workflow-cps">
    <scm class="hudson.plugins.git.GitSCM" plugin="git">
      <configVersion>2</configVersion>
      <userRemoteConfigs><hudson.plugins.git.UserRemoteConfig>
        <url>https://github.com/EmaadMirza/Inventory-devops.git</url>
      </hudson.plugins.git.UserRemoteConfig></userRemoteConfigs>

    <branches><hudson.plugins.git.BranchSpec><name>*/main</name></hudson.plugins.git.BranchSpec></branches>
      <doGenerateSubmoduleConfigurations>false</doGenerateSubmoduleConfigurations>
      <submoduleCfg class="empty-list"/><extensions/>
    </scm>
    <scriptPath>Jenkinsfile</scriptPath>
    <lightweight>true</lightweight>
  </definition>
  <disabled>false</disabled>
</flow-definition>
```